Plataforma SmartCollect - Documento de Arquitetura

Versao: 2.1
Data: 2026-03-18
Autor: Equipe de Engenharia - EmDia
Classificacao: Interno - Arquitetura Tecnica

Indice

1. Sumario Executivo
2. Contexto do Sistema (C4 Nivel 1)
3. Diagrama de Containers (C4 Nivel 2)
4. Diagrama de Componentes: SmartCollect (C4 Nivel 3)
5. Diagrama de Componentes: Mesh Integration (C4 Nivel 3)
6. LangGraph: Arquitetura Multi-Agent
7. Kafka: Arquitetura Orientada a Eventos
8. Modelo de Dados: PostgreSQL + PGVector
9. Diagramas de Sequencia
10. Integracao MCP (Model Context Protocol)
11. Deploy e Infraestrutura
12. Concerns Transversais
13. Apendice A: Glossario
14. Apendice B: Evolucao Phoenix
15. Apendice C: Radar Tecnologico

1. Sumario Executivo

SmartCollect e uma plataforma de negociacao de dividas baseada em uma **arquitetura Multi-Agent** orquestrada por LangGraph. A assistente virtual **Diana** (EmDia) opera via WhatsApp e Telegram, coordenando **agentes especializados de IA** -- cada um com raciocinio proprio via Claude (AWS Bedrock), tools dedicados e dominio especifico. Um **Agente Supervisor** central detecta a intencao do usuario e delega para o agente mais adequado: Onboarding, Transacao, Negociacao, Compliance ou FAQ. O Agente de Negociacao, o mais complexo, conduz negociacoes multi-turno com 14+ cenarios, gera acordos e boletos via RaaS, enquanto o Agente de Compliance garante conformidade com a LGPD durante toda a interacao.

Stakeholders e Audiencia

Audiencia	Interesse
CTO / VP Engenharia	Decisoes tecnologicas estrategicas, escalabilidade, custo
Tech Leads / Arquitetos	Fronteiras do sistema, contratos de integracao, fluxos de dados
Desenvolvedores	Estrutura de componentes, APIs, padroes, deploy
Produto / Negocios	Capacidades, cobertura de canais, fluxo de negociacao

Stack Tecnologica

Camada	Tecnologia	Versao
Motor de IA	Python, LangGraph, LangChain	3.12, >=1.1.2, >=1.1.0
Gateway de Integracao	.NET, ASP.NET Core	10.0
Broker de Mensagens	Apache Kafka (Confluent)	7.5.0

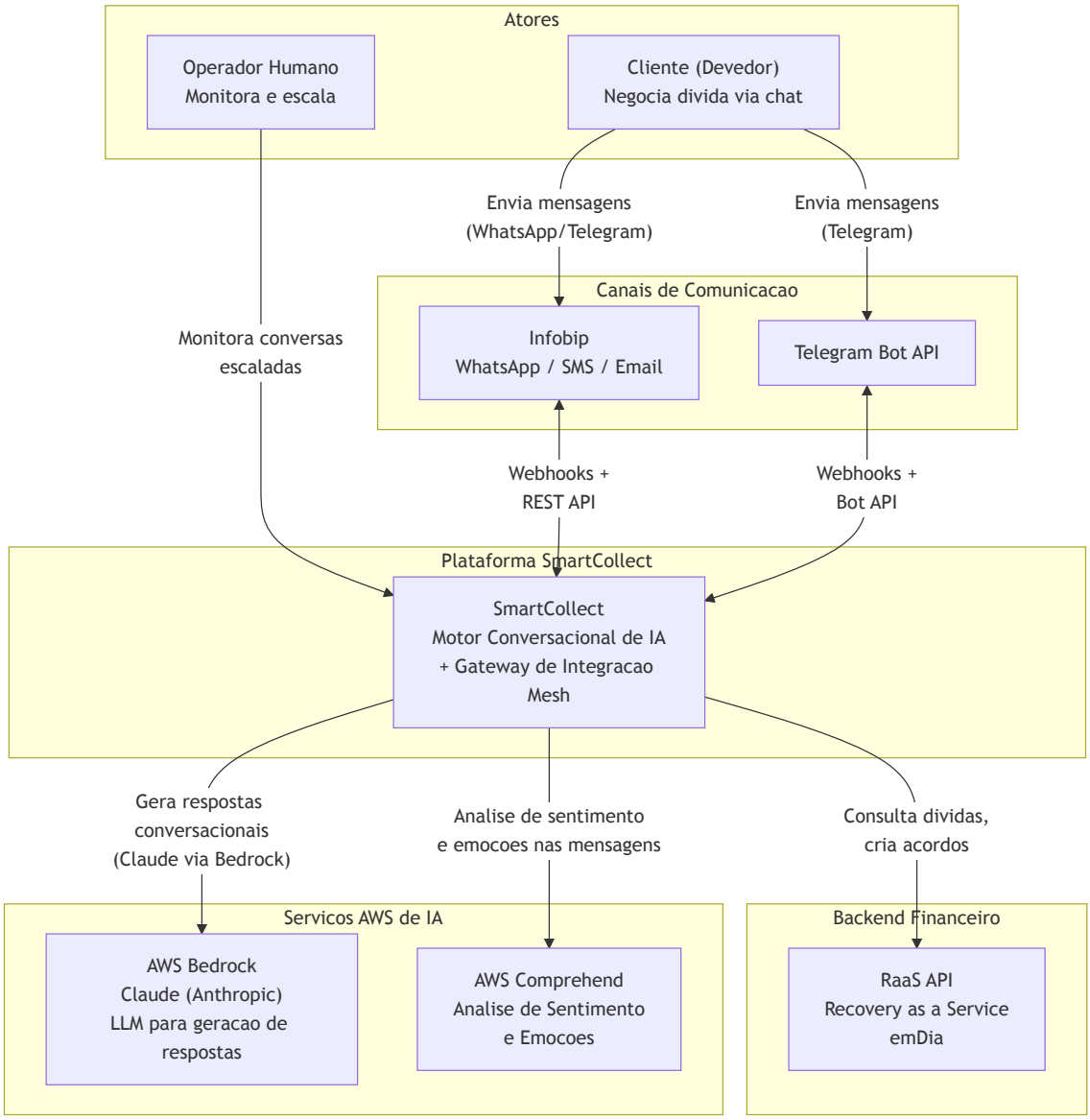
Camada	Tecnologia	Versao
Banco de Dados	PostgreSQL + PGVector	15
Cache / Estado	Redis	7
Canais	Infobip (WhatsApp/SMS/Email), Telegram API	-
Backend Financeiro	RaaS (Recovery as a Service)	v1
IA / ML	AWS Bedrock (Claude), AWS Comprehend	-
Observabilidade	LangSmith, Logging Estruturado, Prometheus	-
Orquestracao de Containers	Docker, Kubernetes (EKS)	-

Componentes da Plataforma

Projeto	Stack	Papel no Repositorio
smartcollect	Python 3.12 / FastAPI / LangGraph	Motor Multi-Agent de IA: Agente Supervisor (orquestrador), Agente de Negociacao (14+ cenarios), Agente de Transacao (MCP/RaaS), Agente de Compliance (LGPD), Agente de FAQ (RAG), consumers/producers Kafka, persistencia PostgreSQL
mesh-smartcollect-integration	.NET 10 / ASP.NET Core	Gateway de integracao: webhooks Infobip, webhooks Telegram, proxy RaaS, servidor de tools MCP, consumers Kafka para notificacoes e acordos
smartcollect-phoenix	Python 3.12 (apenas specs)	Reescrita planejada do smartcollect usando padroes LangGraph v2: Command(update+goto), interrupt(), middleware stack, funcoes puras

2. Contexto do Sistema (C4 Nivel 1)

A plataforma interage com usuarios externos, operadores, provedores de comunicacao, backends financeiros e servicos de IA.

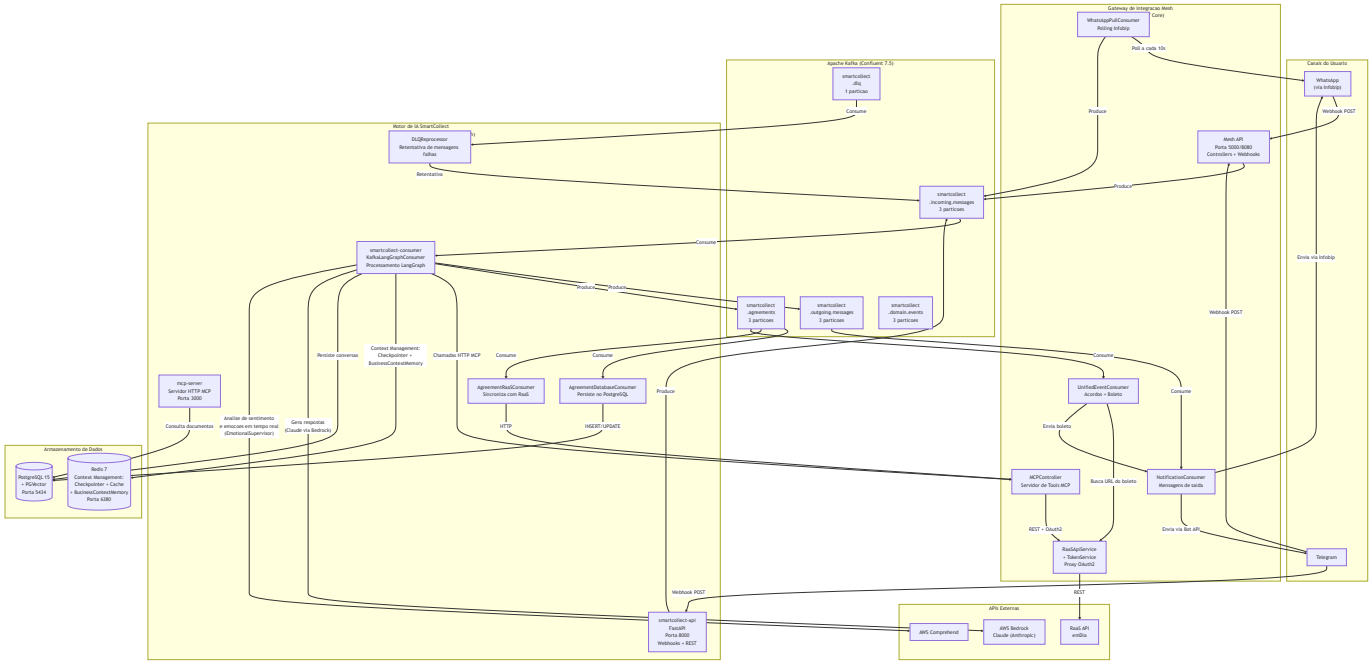


Resumo das Interacoes

Sistema Externo	Protocolo	Direcao	Finalidade
Infobip	HTTPS REST	Bidirecional	Envio e recebimento de WhatsApp/SMS/Email via webhooks
Telegram API	HTTPS REST	Bidirecional	Mensagens do bot Telegram via webhooks
RaaS API	HTTPS REST + OAuth2	Saida	Consulta dividas, busca ofertas, cria acordos, obtem URLs de boleto
AWS Bedrock	HTTPS (AWS SDK)	Saida	LLM unico para geracao de conversas (Claude Anthropic via Bedrock)
AWS Comprehend	HTTPS (AWS SDK)	Saida	Analise de sentimento e emocao nas mensagens do usuario

3. Diagrama de Containers (C4 Nivel 2)

A plataforma e composta por containers de runtime distintos que se comunicam via Kafka e HTTP.

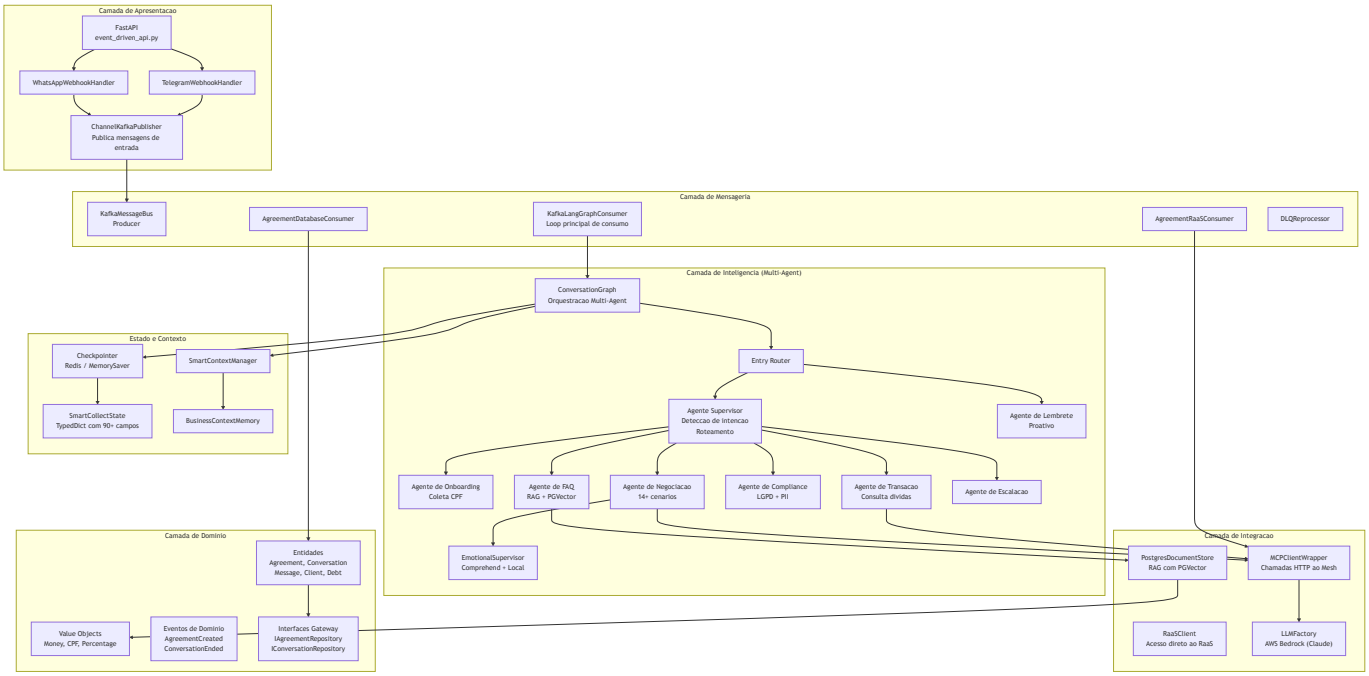


Detalhes dos Containers

Container	Tecnologia	Porta	Perfil Docker	Finalidade
`smartcollect-api`	FastAPI	8000	app	API REST, webhooks Telegram/WhatsApp, endpoints demo
`smartcollect-consumer`	Python	-	app	Consumer Kafka que alimenta mensagens no LangGraph
`mcp-server`	Python (FastAPI)	3000	mcp	Servidor HTTP de tools MCP para integracao com RaaS
`mesh-smartcollect-integration`	ASP.NET Core	5000/8080	-	Gateway de integracao para Infobip, RaaS, MCP, Kafka
`kafka`	Confluent CP	9092	-	Broker de mensagens com 5 topicos
`postgres`	PGVector/PG15	5434	-	Banco de dados principal com busca vetorial
`redis`	Redis 7 Alpine	6380	-	Context Management: Checkpointer LangGraph (estado multi-turn), BusinessContextMemory, cache de sessao
`kafka-ui`	Provectus	8080	-	UI de gerenciamento Kafka
`zookeeper`	Confluent CP	2181	-	Coordenacao Kafka

4. Diagrama de Componentes: SmartCollect (C4 Nivel 3)

Arquitetura interna do motor de IA Python, organizado em 5 camadas.

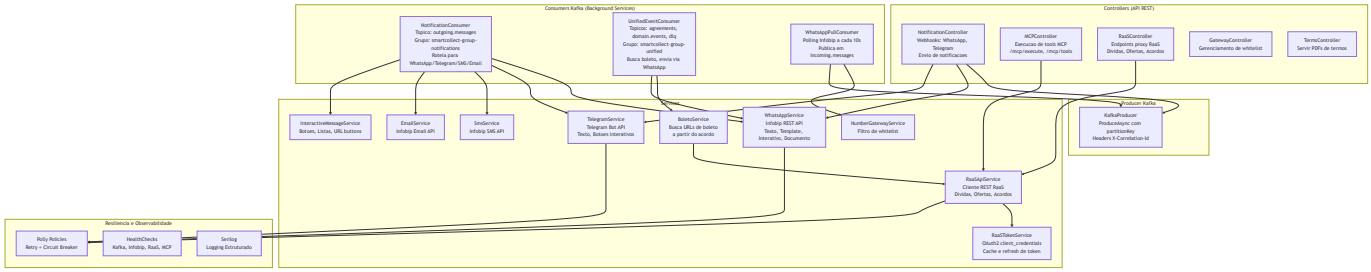


Responsabilidades por Camada

Camada	Responsabilidade	Arquivos Chave
Apresentacao	Endpoints HTTP, tratamento de webhooks, abstracao de canais	`src/api/event_driven_api.py` , `src/channels/telegram/` , `src/channels/whatsapp/`
Inteligencia (Multi-Agent)	Orquestracao Multi-Agent via LangGraph, agentes especializados (Supervisor, Negociacao, Transacao, Compliance, FAQ, Onboarding, Lembrete, Escalacao), roteamento de intencoes	`src/presentation/graph/conversation_graph.py` , `src/agents/` , `src/core/supervisor/`
Integracao	Adaptadores de servicos externos, fabrica de LLM, repositorio de documentos	`src/mcp/` , `src/llm/` , `src/infrastructure/knowledge/`
Mensageria	Producers, consumers, schemas Kafka, DLQ	`src/messaging/kafka/` , `scripts/kafka_langgraph_consumer.py`
Dominio	Entidades de negocio, value objects, eventos de dominio, interfaces	`src/domain/`

5. Diagrama de Componentes: Mesh Integration (C4 Nivel 3)

Arquitetura interna do gateway de integracao .NET.



Resumo de Endpoints

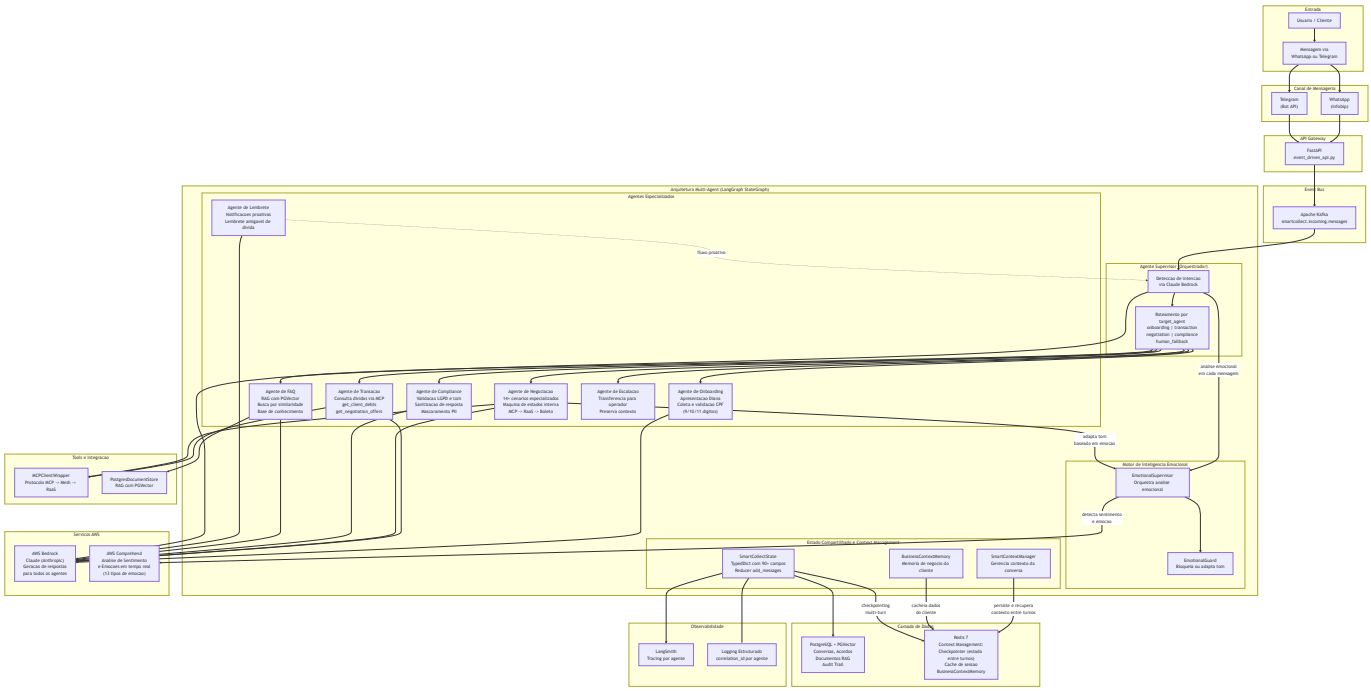
Controller	Endpoints Principais	Finalidade
NotificationController	`POST /api/notification` , `POST /api/notification/webhook/whatsapp` , `POST /api/notification/webhook/telegram`	Roteamento de mensagens entrada/saida
MCPController	`GET /mcp/tools` , `POST /mcp/execute` , `POST /mcp/get_client_debts` , `POST /mcp/get_negotiation_offers` , `POST /mcp/create_arrangement`	Interface de tools MCP para os Agentes de Transacao e Negociacao
RaaSController	`POST /get_client_debts` , `POST /get_negotiation_offers` , `POST /create_arrangement` , `GET /arrangement_status/{id}`	Proxy direto para RaaS
GatewayController	`GET /api/gateway/numbers` , `POST /api/gateway/numbers` , `PUT /api/gateway/status`	Whitelist de numeros WhatsApp
TermsController	`GET /api/terms/use/{filename}`	Servir PDFs de termos e condicoes

6. LangGraph: Arquitetura Multi-Agent

SmartCollect implementa uma **arquitetura Multi-Agent** orquestrada por LangGraph, seguindo o padrao **Supervisor Multi-Agent** documentado pela LangChain. Ao inves de simples funcoes executando em nodes de um grafo, a plataforma opera com **agentes especializados de IA** -- cada um com raciocinio proprio via Claude (AWS Bedrock), tools dedicados, system prompt exclusivo e dominio especifico de atuacao.

Um **Agente Supervisor** central recebe cada mensagem, detecta a intencao do usuario via LLM e delega para o agente especializado mais adequado. Cada agente herda de `BaseAgent` , que fornece capacidade autonoma de geracao de respostas, composicao de prompts e deteccao de contexto.

6.1 Padrao Multi-Agent: Supervisor com Agentes Especializados



6.2 Catalogo de Agentes Especializados

Cada agente herda de `BaseAgent` e possui: system prompt exclusivo, cadeia de geracao via LLM (Claude/Bedrock), acesso a tools especificos e logica de dominio propria.

Agente	Classe	Especialidade	Tools / Servicos	Autonomia
Supervisor	`create_supervisor_node`	Deteccao de intencao, roteamento, contexto emocional	AWS Bedrock (Claude), EmotionalSupervisor (AWS Comprehend), SmartContextManager (Redis)	Decide qual agente atende cada mensagem; classifica entre 5 intencoes; analisa emocao via Comprehend
Onboarding	`OnboardingAgent`	Apresentacao da Diana, coleta e validacao de CPF	AWS Bedrock (Claude)	Extrai CPF de 9/10/11 digitos, valida formato, inicia identificacao
Transacao	`TransactionAgent`	Consulta de dividas e situacao financeira	MCPClientWrapper (`get_client_debts` , `get_negotiation_offers`)	Invoca tools MCP para buscar dados do RaaS via Mesh
Negociacao	`NegotiationAgent`	Negociacao multi-turno completa	AWS Bedrock (Claude), MCPClientWrapper, EmotionalSupervisor (AWS Comprehend)	Sub-grafo com 14+ cenarios, maquina de estados, criacao de acordos e boletos; adapta tom baseado em emocao
Compliance	`ComplianceAgent`	Validacao LGPD, tom, sanitizacao	AWS Bedrock (Claude)	Valida resposta antes do envio, mascara PII, bloqueia conteudo inadequado
FAQ	`FAQAgent`	Respostas baseadas em conhecimento (RAG)	PostgresDocumentStore + PGVector	Busca por similaridade vetorial, enriquece prompt com documentos relevantes
Lembrete	`DebtReminderAgent`	Notificacoes proativas de divida	AWS Bedrock (Claude)	Gera lembretes amigaveis com persona Diana, pitch persuasivo
Escalacao	`create_human_fallback_node`	Transferencia para operador humano	Nenhum	Preserva contexto completo da conversa para o operador

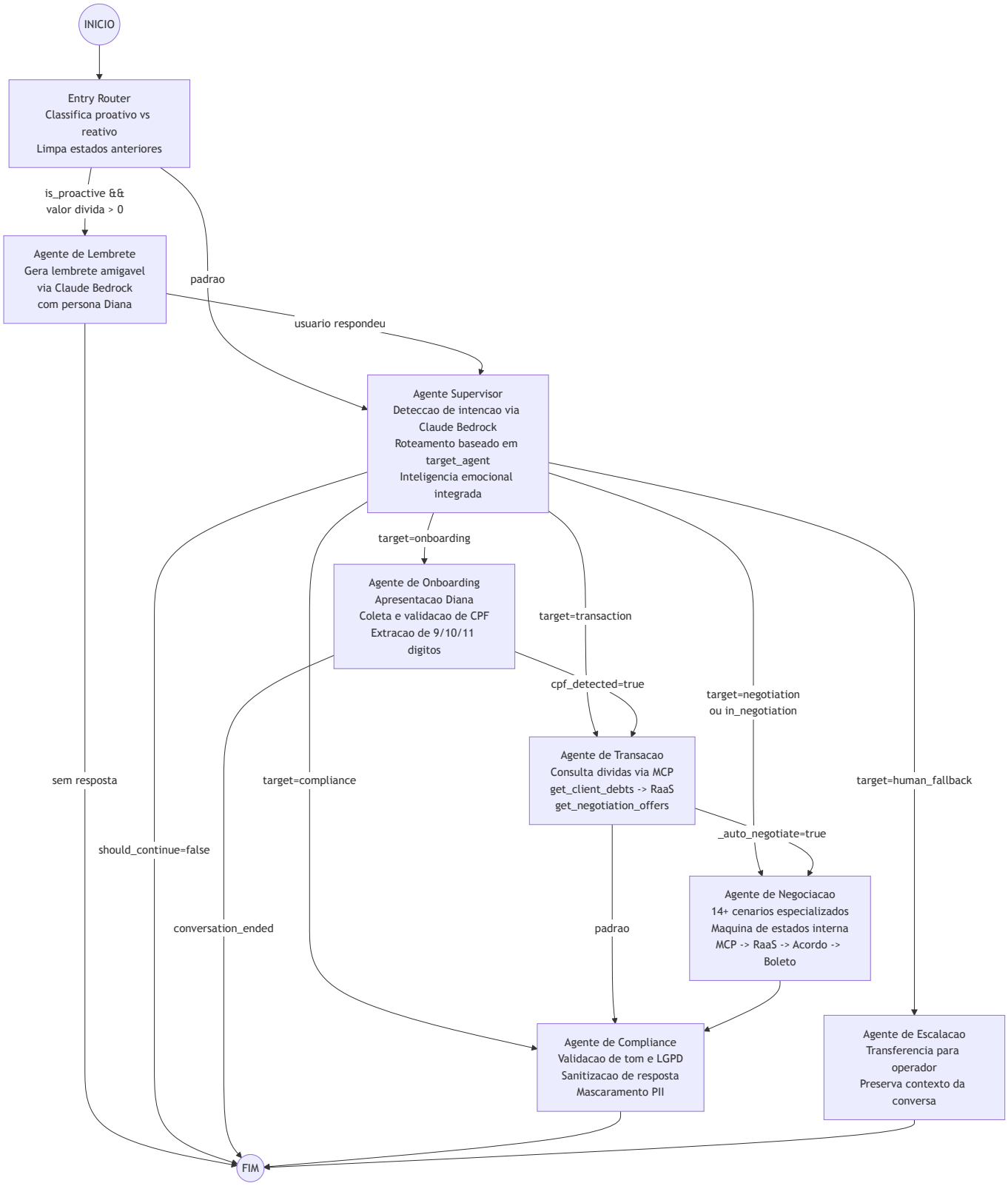
Fontes: `src/agents/base_agent.py` , `src/agents/` , `src/core/supervisor/routing_config.py`

6.3 Por que Agentes e nao Nodes

Caracteristica	Node simples	Agente especializado (SmartCollect)
Raciocinio	Logica deterministica	Raciocinio via LLM (Claude/Bedrock) com system prompt proprio
Tools	Nenhum ou compartilhado	Tools dedicados por dominio (MCP, RAG, Comprehend)
Prompt	Nao possui	System prompt exclusivo com persona e instrucoes especificas
Estado interno	Nao possui	Maquina de estados interna (Negociacao tem 14+ estados)
Autonomia	Executa logica fixa	Toma decisoes baseadas em contexto, emocao e historico
Heranca	Funcao pura	`BaseAgent` com ResponseGenerator, PromptComposer, ContextDetector

6.4 Topologia do Grafo de Orquestracao

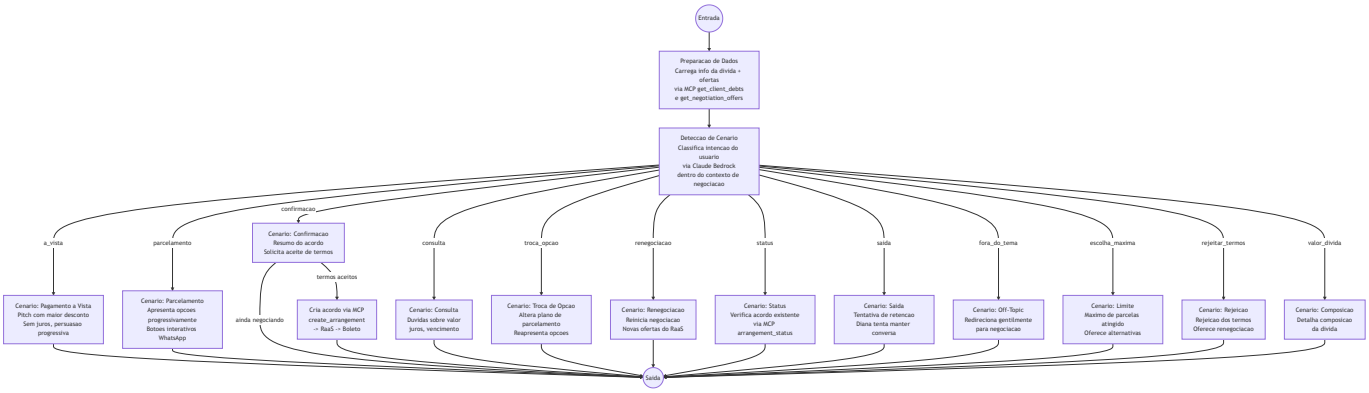
O LangGraph `StateGraph` funciona como **infraestrutura de orquestracao** dos agentes. Cada ponto no grafo encapsula um agente especializado com `ConversationGraphBuilder` montando a topologia com injecao de dependencias.



Fonte: `src/presentation/graph/conversation_graph.py`

6.5 Agente de Negociacao: Sub-Grafo com 14+ Cenarios

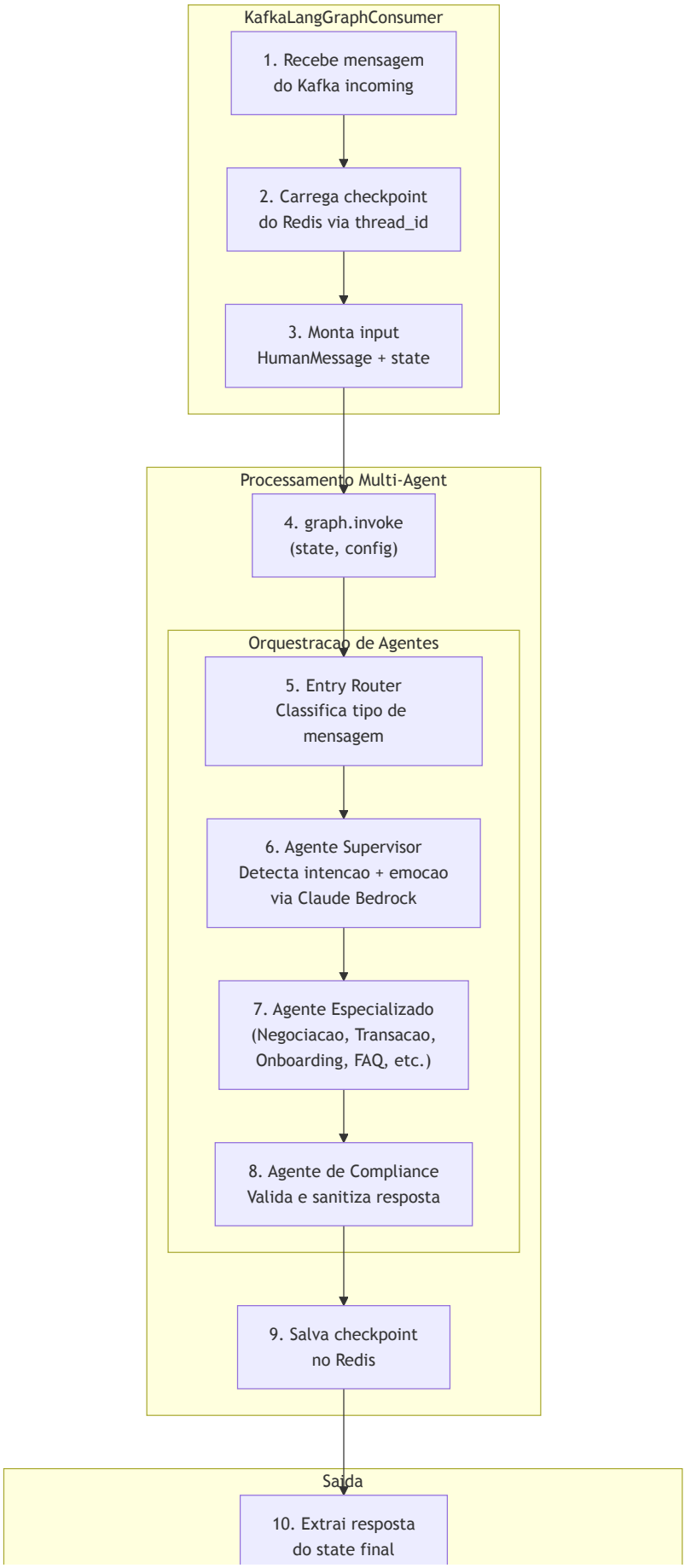
O Agente de Negociacao e o mais complexo da plataforma. Internamente opera como uma maquina de estados (`NegotiationStateMachine`) com 14+ cenarios roteados via `detect_scenario` (classificacao de intencao via Claude Bedrock dentro do contexto de negociacao). Cada cenario pode invocar tools MCP para consultar o RaaS e gerar respostas persuasivas.

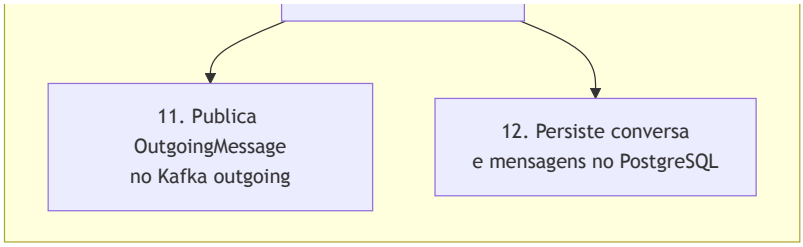


Fonte: `src/agents/negotiation/negotiation_graph.py`

6.6 Ciclo de Vida de uma Mensagem

O diagrama abaixo mostra como uma unica mensagem do usuario e processada pela arquitetura Multi-Agent:


KafkaLangGraphConsumer
1. Recebe mensagem do Kafka incoming
2. Carrega checkpoint do Redis via thread_id
3. Monta input HumanMessage + state
Processamento Multi-Agent
4. graph.invoke (state, config)
Orquestracao de Agentes
5. Entry Router Classifica tipo de mensagem
6. Agente Supervisor Detecta intencao + emocao via Claude Bedrock
7. Agente Especializado (Negociacao, Transacao, Onboarding, FAQ, etc.)
8. Agente de Compliance Valida e sanitiza resposta
9. Salva checkpoint no Redis
Saida
10. Extrai resposta do state final



6.7 Estado Compartilhado entre Agentes

Todos os agentes compartilham o `SmartCollectState`, um `TypedDict` com 90+ campos organizados em sub-estados. Cada agente le e escreve nos campos relevantes ao seu dominio:

Sub-Estado	Campos Principais	Agentes que Utilizam
`ClientState`	`cpf`, `name`, `display_name`, `onboarding_complete`	Onboarding, Supervisor
`NegotiationState`	`in_negotiation`, `agreement_created`, `option_chosen`, `terms_shown`, `summary_shown`	Negociacao, Supervisor
`FlowControlState`	`should_continue`, `target_agent`, `user_intent`, `guardrail_blocked`, `user_wants_to_end`	Supervisor (escrita), todos (leitura)
`ResponseState`	`supervisor_response`, `negotiation_response`, `transaction_response`, `fallback_message`	Cada agente escreve sua propria resposta
`ComplianceState`	`result`, `warning`	Compliance
`TransactionState`	`result`, `payment_confirmed`	Transacao
`ConversationMetadata`	`conversation_id`, `is_proactive`, `bot_introduced`, `summary`	Todos

Campos adicionais relevantes para o Agente de Negociacao:

Campo	Tipo	Finalidade
`messages`	`Annotated[Sequence[BaseMessage], add_messages]`	Historico da conversa (reducer append-only, gerenciado pelo checkpointer)
`negotiation_offers`	`Dict[str, Any]`	Ofertas carregadas do RaaS via MCP
`selected_offer_snapshot`	`Dict[str, Any]`	Snapshot da oferta selecionada pelo usuario
`best_offer`	`Dict[str, Any]`	Melhor oferta disponivel (a vista, maior desconto)
`emotional_analysis`	`Dict[str, Any]`	Resultado da analise emocional do AWS Comprehend
`negotiation_stage`	`str`	Estagio atual da maquina de estados de negociacao
`interactive_type`	`Literal["button", "list", "url_button", "call_button"]`	Tipo de mensagem interativa WhatsApp
`interactive_buttons`	`List[Dict]`	Definicoes de botoes interativos (max 3)
`interactive_sections`	`List[Dict]`	Secoes de lista interativa (max 10)

6.8 Checkpointing e Persistencia de Estado

O LangGraph usa checkpointing para manter o estado compartilhado entre turnos (multi-turn). Cada turno do usuario resulta em uma invocacao completa do grafo de agentes, e o estado e salvo no final.

Aspecto	Detalhe
Chave de thread	`conversation_id` (UUID)

Aspecto	Detalhe
Backend dev	`MemorySaver` (em memoria, perde estado ao reiniciar)
Backend producao	Redis via `RedisSaver` (persistente, TTL configuravel)
Reducer	`add_messages` no campo `messages` (append-only)
Demais campos	Last-write-wins (ultima escrita prevalece)
Compilacao	`workflow.compile(checkpointer=checkpointer)`

6.9 Integracao com AWS Bedrock

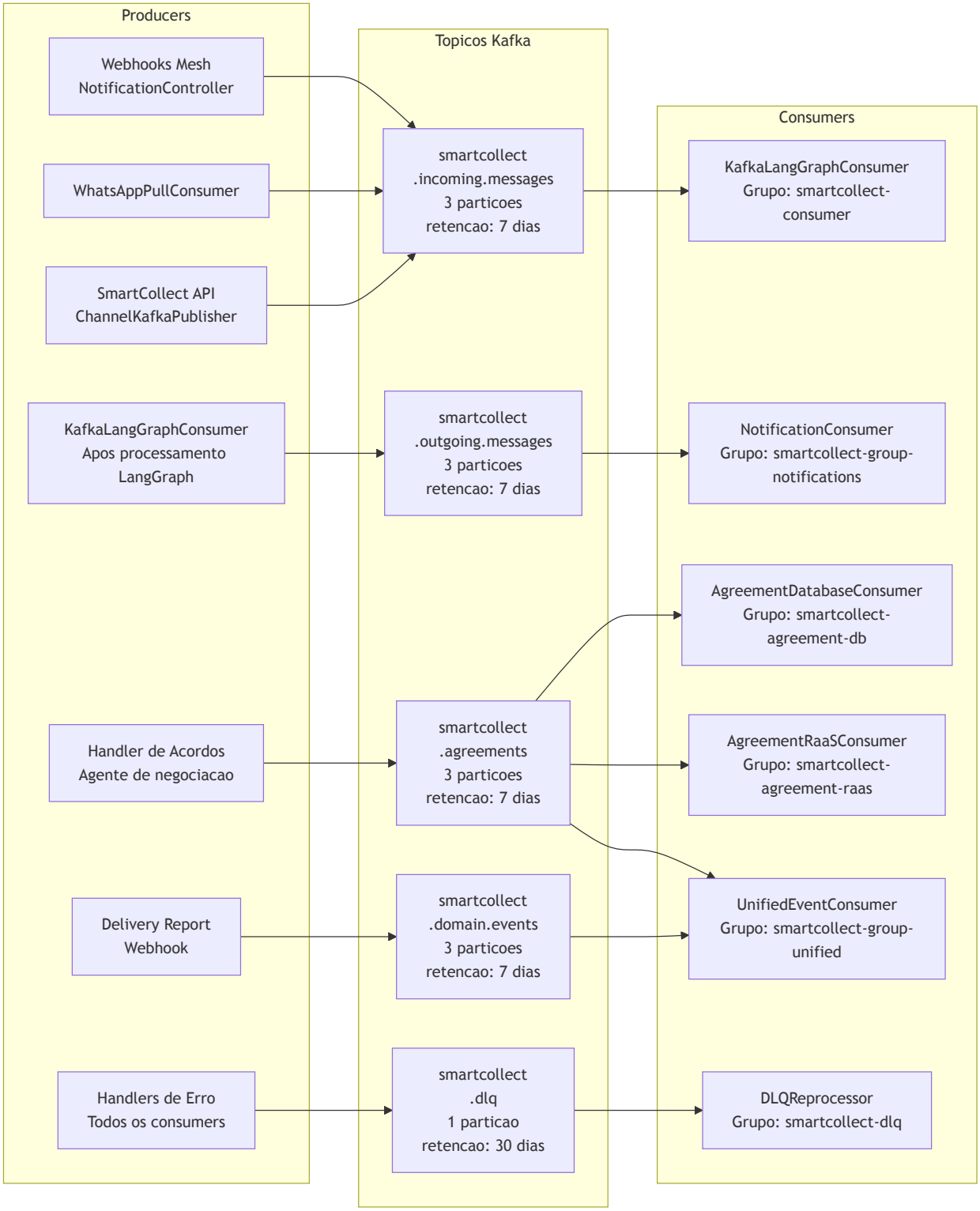
Todos os agentes compartilham o mesmo provedor LLM via `LLMFactory`, que abstrai a criacao do cliente Bedrock:

Aspecto	Detalhe
Provedor	AWS Bedrock
Modelo	Claude (Anthropic)
SDK	`langchain-aws` (ChatBedrock)
Regiao	`us-east-1` (configuravel via `AWS_REGION`)
Autenticacao	AWS SDK (IAM credentials ou instance profile)
Agentes que chamam LLM	Supervisor, Onboarding, Lembrete, Transacao, Negociacao (14+ cenarios), Compliance, FAQ

Fonte: `src/core/state/smartcollect_state.py`, `src/presentation/graph/conversation_graph.py`, `src/agents/base_agent.py`

7. Kafka: Arquitetura Orientada a Eventos

7.1 Mapa de Topicos



7.2 Detalhes dos Topicos

Topico	Particoes	Retencao	Chave de Particao	Grupos de Consumo
`smartcollect.incoming.messages`	3	7 dias	`conversation_id`	`smartcollect-consumer`
`smartcollect.outgoing.messages`	3	7 dias	`conversation_id`	`smartcollect-group-notifications`
`smartcollect.agreements`	3	7 dias	`agreement_id`	`smartcollect-agreement-db` , `smartcollect-agreement-raas` , `smartcollect-group-unified`
`smartcollect.domain.events`	3	7 dias	`event_id`	`smartcollect-group-unified`
`smartcollect.dlq`	1	30 dias	`original_topic`	`smartcollect-dlq`

7.3 Schemas de Mensagens

OutgoingMessage (smartcollect.outgoing.messages)

Campo	Tipo	Obrigatorio	Descricao
`channel`	string	Sim	Canal de destino (telegram, whatsapp)
`to`	string	Sim	Identificador do destinatario
`chat_id`	string	Sim	ID do chat especifico do canal
`text`	string	Sim	Conteudo textual da mensagem
`conversation_id`	string	Nao	UUID da conversa
`correlation_id`	string	Nao	Correlacao ponta-a-ponta
`agent`	string	Nao	Agente que gerou a resposta
`interactive_type`	enum	Nao	button, list, url_button, call_button
`interactive_buttons`	array	Nao	Definicoes de botoes (max 3)
`interactive_sections`	array	Nao	Secoes de lista (max 10)
`media_url`	string	Nao	URL de midia anexa

Fonte: `src/messaging/kafka/schemas.py`

AgreementCreationRequest (smartcollect.agreements)

Campo	Tipo	Obrigatorio	Descricao
`event_id`	UUID	Sim	Identificador unico do evento
`event_type`	string	Sim	`agreement_creation_requested`
`conversation_id`	string	Sim	Conversa de origem
`client_id`	string	Sim	CPF/CNPJ
`debt_id`	string	Sim	Identificador da divida
`option`	object	Sim	Parcelas, valor, total
`debt_data`	object	Sim	Informacoes completas da divida
`correlation_id`	string	Nao	Rastreamento ponta-a-ponta

7.4 Fluxo do Correlation ID

Cada mensagem carrega um `correlation_id` que flui ponta-a-ponta:

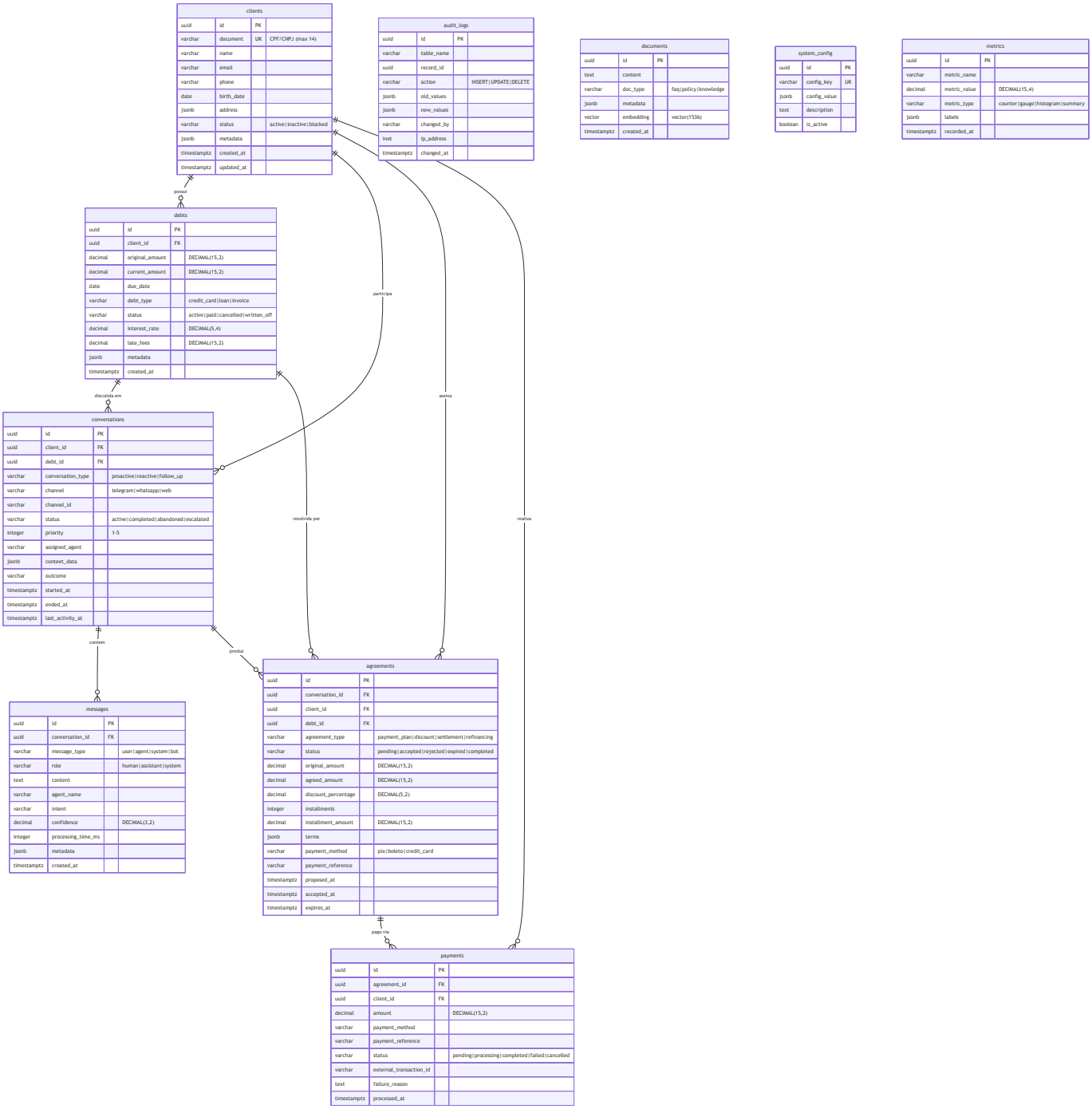
```
Webhook (gera UUID) --> Header Kafka X-Correlation-Id --> KafkaLangGraphConsumer
    --> Estado LangGraph --> Header HTTP MCPClient --> Mesh --> RaaS
    --> Resposta --> Kafka outgoing (mesmo correlation_id) --> Logs
```

Todas as entradas de log estruturado incluem `correlation_id` para rastreamento distribuido entre os servicos SmartCollect e Mesh.

Fonte: `src/core/messaging/correlation_manager.py`

8. Modelo de Dados: PostgreSQL + PGVector

8.1 Diagrama Entidade-Relacionamento



8.2 Estrategia de Indices

Tabela	Indice	Tipo	Finalidade
`clients`	`idx_clients_document`	B-tree	Busca por CPF/CNPJ
`clients`	`idx_clients_name`	GIN (pg_trgm)	Busca fuzzy por nome
`debts`	`idx_debts_client_id`	B-tree	Busca de dividas por cliente

Tabela	Indice	Tipo	Finalidade
`debts`	`idx_debts_status`	B-tree	Filtragem de dividas ativas
`messages`	`idx_messages_content`	GIN (pg_trgm)	Busca full-text em mensagens
`documents`	`idx_documents_embedding`	IVFFlat (cosseno)	Busca por similaridade vetorial para RAG
`agreements`	`idx_agreements_status`	B-tree	Filtragem por status do acordo
`audit_logs`	`idx_audit_logs_table_record`	B-tree (composto)	Busca na trilha de auditoria

8.3 Configuracao PGVector

- Extensao: `pgvector` no PostgreSQL 15
- Dimensao do embedding: 1536 (AWS Bedrock Embeddings)
- Tipo de indice: IVFFlat com similaridade cosseno (`vector_cosine_ops`)
- Parametro lists: 100 (para ~10K-100K documentos)
- Utilizado pelo `PostgresDocumentStore` para recuperacao de documentos RAG

8.4 Views do Banco de Dados

View	Finalidade
`active_conversations`	Dashboard de conversas ativas com contagem de mensagens
`agreement_summary`	Relatorio de todos os acordos com detalhes do cliente e canal

8.5 Row Level Security

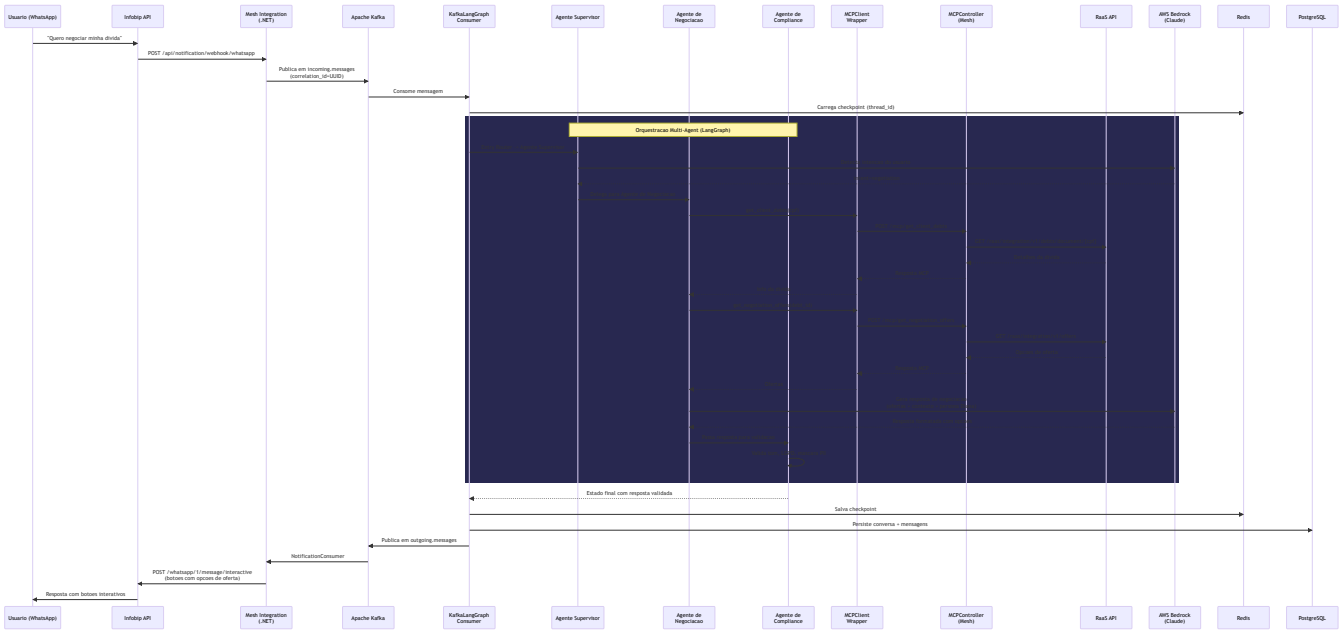
RLS esta habilitado em todas as tabelas principais (`clients` , `debts` , `conversations` , `messages` , `agreements` , `payments`) com politicas permissivas padrao, pronto para isolamento de tenant quando necessario.

Fonte: `database/schema_postgresql.sql` , `database/migrations/002_add_pgvector.sql`

9. Diagramas de Sequencia

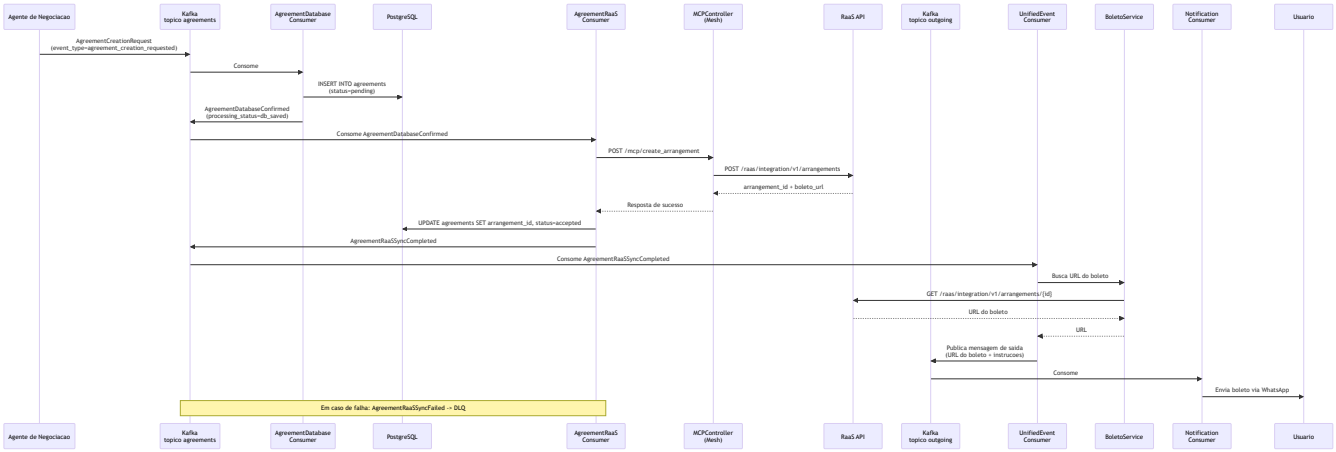
9.1 Fluxo Principal de Negociacao

Fluxo ponta-a-ponta desde a mensagem do usuario ate a entrega da resposta, mostrando a orquestracao Multi-Agent.

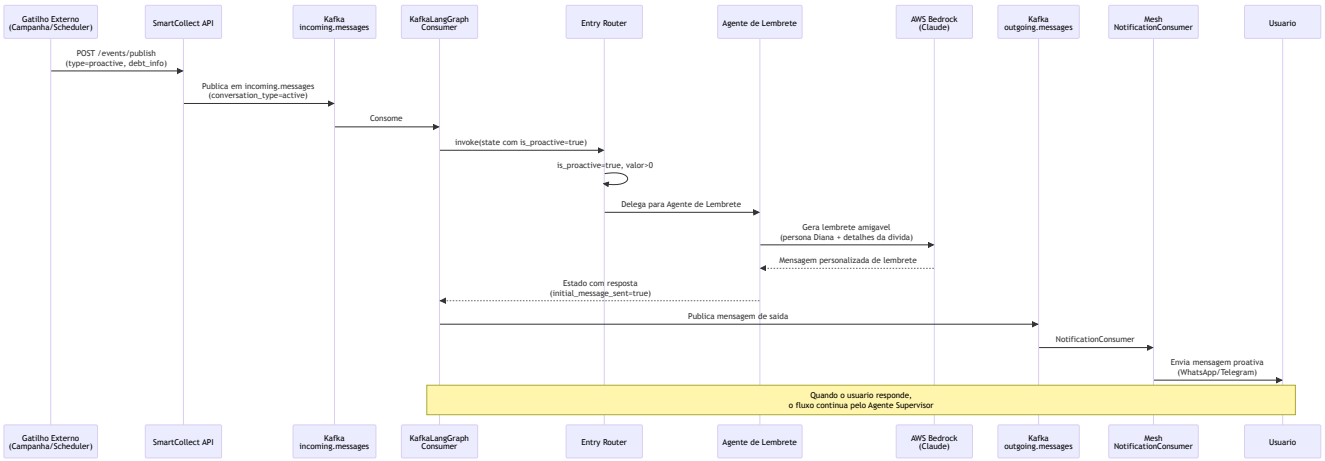


9.2 Fluxo de Criacao de Acordo

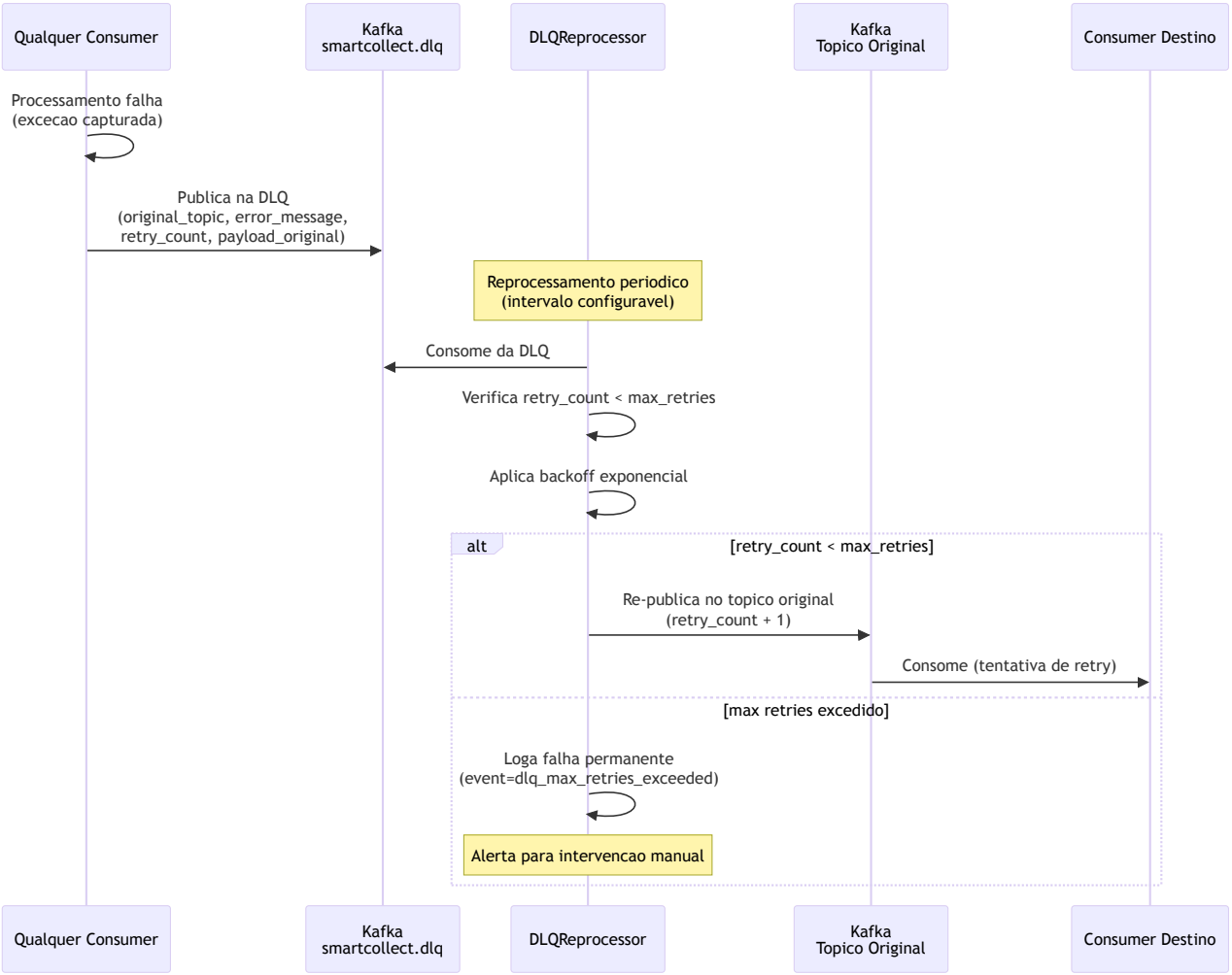
Saga baseada em coreografia para criacao de acordo entre multiplos consumers.



9.3 Fluxo Proativo de Lembrete de Divida



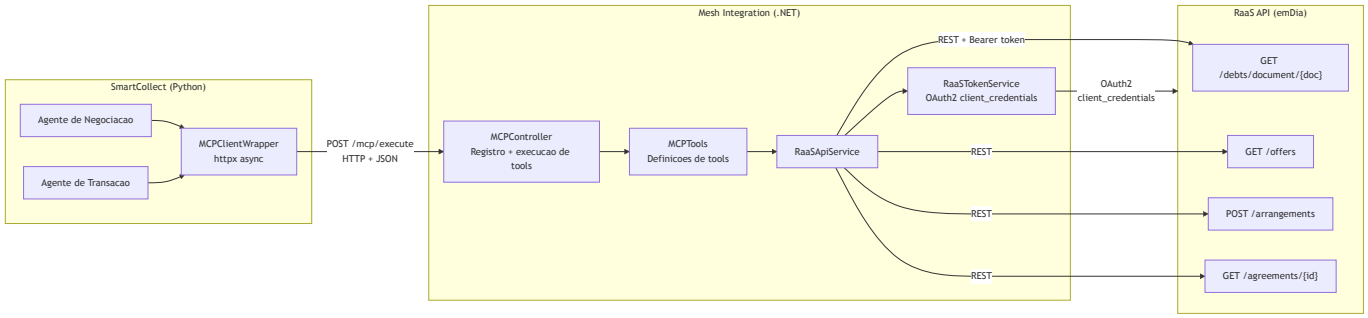
9.4 Fluxo de DLQ e Recuperacao de Erros



10. Integracao MCP (Model Context Protocol)

10.1 Arquitetura

A camada MCP fornece uma interface padronizada de tools entre os agentes de IA do SmartCollect e o backend financeiro RaaS, intermediada pelo gateway de integracao Mesh.



10.2 Catalogo de Tools MCP

Tool	Metodo	Endpoint Mesh	Endpoint RaaS	Descrica
`get_client_debts`	POST	`/mcp/get_client_debts`	`GET /raas/integration/v1/debts/document/{document}`	Recupera todas as dividas de um CPF/CNP
`get_negotiation_offers`	POST	`/mcp/get_negotiation_offers`	`GET /raas/integration/v1/offers`	Obtem ofertas de negociaca disponivei para uma divida
`create_arrangement`	POST	`/mcp/create_arrangement`	`POST /raas/integration/v1/arrangements`	Cria um acordo de pagament (arrangem
`get_agreement`	POST	`/mcp/execute`	`GET /api/v1/agreements/{agreement_id}`	Recupera detalhes d acordo
`get_client_agreements`	POST	`/mcp/execute`	`GET /api/v1/agreements/client/{client_id}`	Lista todo os acordo de um clie
`update_agreement_status`	POST	`/mcp/execute`	`PUT /api/v1/agreements/{agreement_id}/status`	Atualiza status do acordo
`arrangement_status`	GET	`/arrangement_status/{id}`	`GET /raas/integration/v1/arrangements/{id}`	Verifica status do acordo (in URL do boleto)
`product_details`	GET	`/product-details/{debtId}`	`GET /raas/integration/v1/debts/{debt_id}/details`	Obtem informaco detalhada do produt da divida

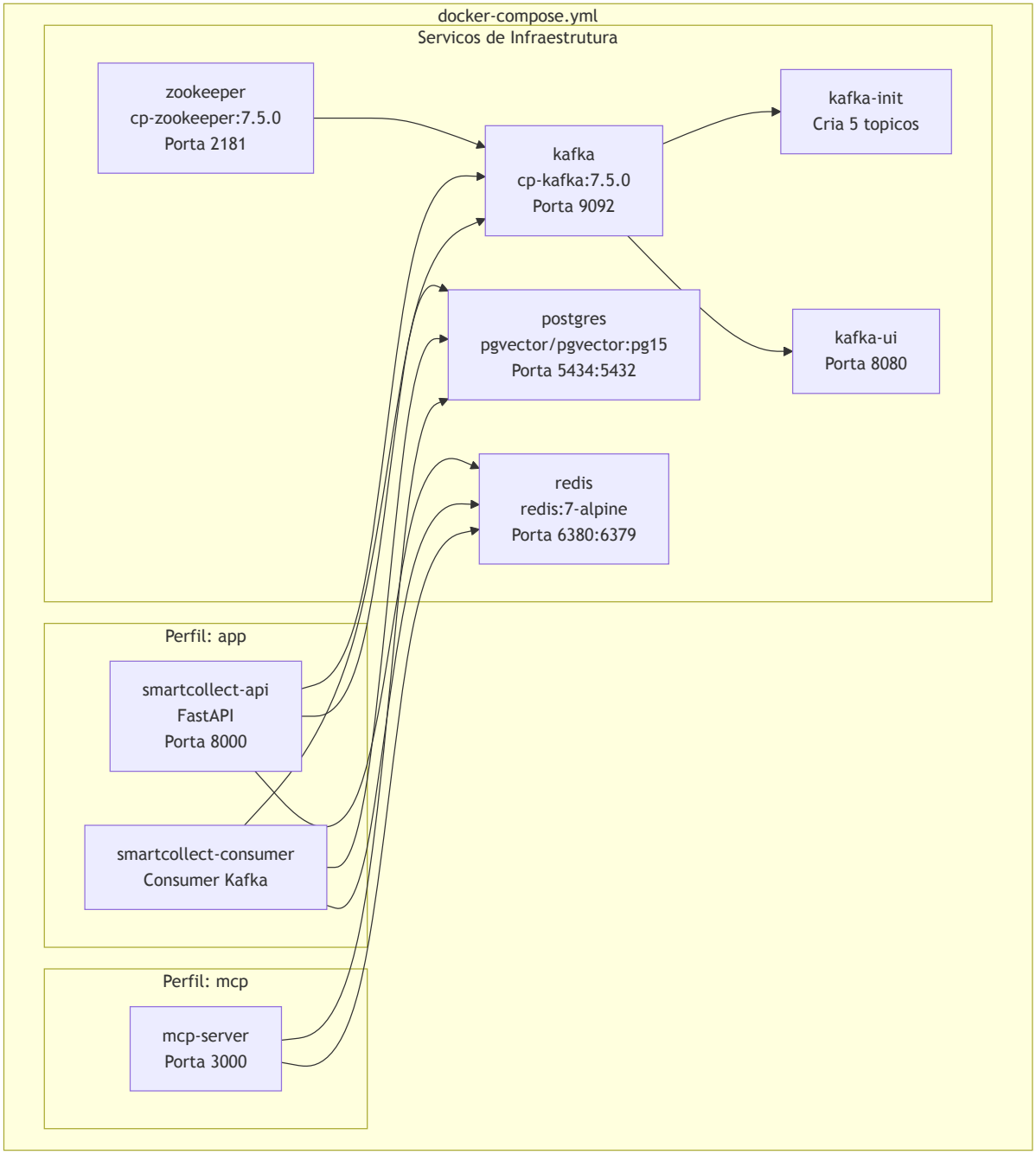
10.3 Fluxo de Autenticacao

```
MCPClientWrapper -> POST /mcp/get_client_debts
    -> MCPController -> RaaSApiService
        -> RaaSTokenService
            -> POST https://apis.dv.euemdia.com.br/oauth2/token/client_credentials
               (grant_type=client_credentials, client_id=sciensa, realm=emdia-core)
            <- access_token (cacheado ate expirar)
        -> GET /raas/integration/v1/debts/document/{cpf}
           (Authorization: Bearer {token})
        <- Dados da divida
    <- Resposta MCP JSON
```

Fonte: `src/mcp/mcp_server.py` , `src/agents/negotiation/mcp_client.py` , `mesh-smartcollect-integration/MCP/` , `mesh-smartcollect-integration/services/RaaSApiService.cs`

11. Deploy e Infraestrutura

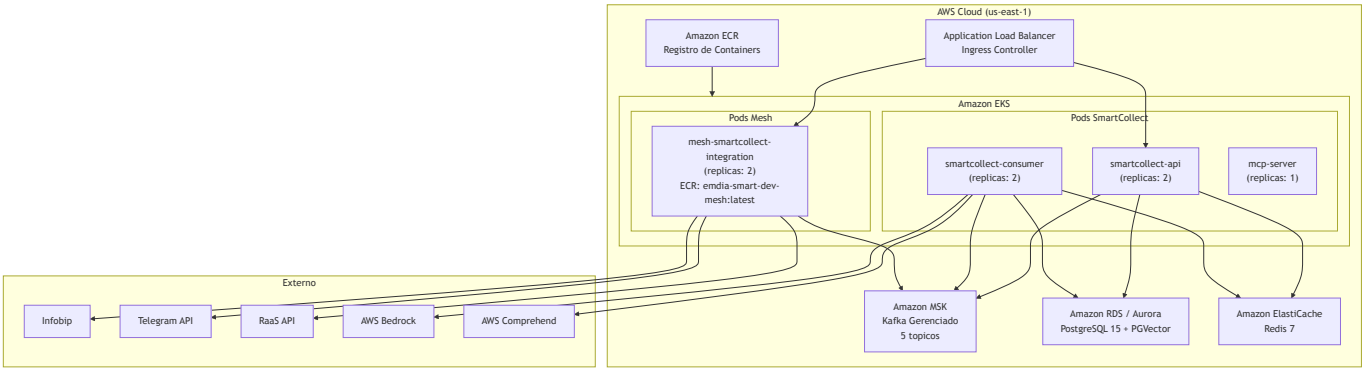
11.1 Desenvolvimento Local (Docker Compose)



Comandos:

Comando	Finalidade
`docker-compose up -d`	Inicia apenas infraestrutura (Redis, Postgres, Kafka)
`docker-compose --profile app up -d`	Inicia infraestrutura + aplicacao
`docker-compose --profile mcp up -d`	Inicia infraestrutura + servidor MCP

11.2 Arquitetura de Producao (AWS)



11.3 Recursos Kubernetes (Mesh)

Recurso	Arquivo	Detalhes
Deployment	`ci-cd/deployment.yaml`	2 replicas, 512Mi-1Gi memoria, 250m-500m CPU
Service	`ci-cd/service.yaml`	ClusterIP na porta 8080
Ingress	`ci-cd/ingress.yaml`	ALB Ingress Controller
ConfigMap	`ci-cd/configmap.yaml`	Configuracoes Kafka, Infobip, RaaS, Telegram, MCP
Secrets	`ci-cd/secrets.yaml.template`	Chaves de API, senhas, tokens
Namespace	`ci-cd/namespace.yaml`	Namespace `smartcollect`

11.4 Mapa de Portas

Servico	Porta Local	Porta Container	Protocolo
SmartCollect API	8000	8000	HTTP
Servidor MCP	3000	3000	HTTP
Mesh Integration	5000	8080	HTTP
PostgreSQL	5434	5432	TCP
Redis	6380	6379	TCP
Kafka	9092	9092	TCP
Kafka (interno)	-	29092	TCP
Zookeeper	2181	2181	TCP
Kafka UI	8080	8080	HTTP

11.5 Variaveis de Ambiente

Categoria	Variaveis	Servico
Banco de Dados	`POSTGRES_HOST` , `POSTGRES_PORT` , `POSTGRES_DB` , `POSTGRES_USER` , `POSTGRES_PASSWORD` , `DATABASE_URL`	SmartCollect
Redis	`REDIS_HOST` , `REDIS_PORT` , `REDIS_URL`	SmartCollect
Kafka	`KAFKA_BOOTSTRAP_SERVERS` , `KAFKA_CONSUMER_GROUP` , `KAFKA_ENABLED`	SmartCollect, Mesh
AWS	`AWS_REGION` , `AWS_ACCESS_KEY_ID` , `AWS_SECRET_ACCESS_KEY`	SmartCollect

Categoria	Variaveis	Servico
LLM	`LLM_PROVIDER` , `LLM_MODEL` , `BEDROCK_MODEL_ID`	SmartCollect
RaaS	`RAAS_BASE_URL` , `RAAS_API_KEY` , `RAAS_TOKEN_URL` , `RAAS_CLIENT_ID` , `RAAS_CLIENT_SECRET`	SmartCollect, Mesh
MCP	`MCP_SERVER_URL`	SmartCollect
Infobip	`Infobip__ApiKey` , `Infobip__BaseUrl` , `Infobip__WhatsappNumber`	Mesh
Telegram	`TELEGRAM_BOT_TOKEN` , `Telegram__BotToken`	SmartCollect, Mesh
Comprehend	`COMPREHEND_ENABLED` , `COMPREHEND_LANGUAGE`	SmartCollect
Observabilidade	`LANGSMITH_API_KEY` , `LANGSMITH_PROJECT` , `LOG_LEVEL`	SmartCollect

12. Concerns Transversais

12.1 Observabilidade

Logging Estruturado

Todos os servicos usam logging estruturado em ASCII puro com formato key=value:

```
event=message_processed conversation_id=abc-123 correlation_id=def-456 agent=negotiation duration_ms=1200 intent=negotiation
```

Regras:

- Apenas ASCII puro, sem emojis ou simbolos Unicode
- Formato key=value para parsing por maquinas
- `correlation_id` em todas as entradas de log para rastreamento distribuido
- SmartCollect: modulo `logging` do Python
- Mesh: Serilog com saida estruturada

Metricas

Metrica	Tipo	Origem
`conversations_total`	counter	SmartCollect
`agreements_created_total`	counter	SmartCollect
`message_processing_duration_seconds`	histogram	SmartCollect
`kafka_consumer_lag`	gauge	Kafka
`llm_request_duration_seconds`	histogram	SmartCollect
`raas_request_duration_seconds`	histogram	Mesh

Health Checks

Servico	Endpoint	Verificacoes
SmartCollect API	`GET /health`	Status da aplicacao
SmartCollect API	`GET /status`	Status do sistema com detalhes dos componentes
Mesh Integration	`GET /health/live`	Sonda de liveness
Mesh Integration	`GET /health/ready`	Readiness: conectividade Kafka, Infobip, RaaS, MCP

Servico	Endpoint	Verificacoes
Servidor MCP	`GET /health`	Status do servidor MCP

Integracao LangSmith

Quando habilitado (`LANGSMITH_TRACING=true`), todas as invocacoes do grafo Multi-Agent sao rastreadas no LangSmith para:

- Visualizacao do fluxo de orquestracao entre agentes
- Inspecao de chamadas LLM por agente (entrada/saida/tokens)
- Analise de latencia por agente especializado
- Rastreamento e depuracao de erros na cadeia de delegacao

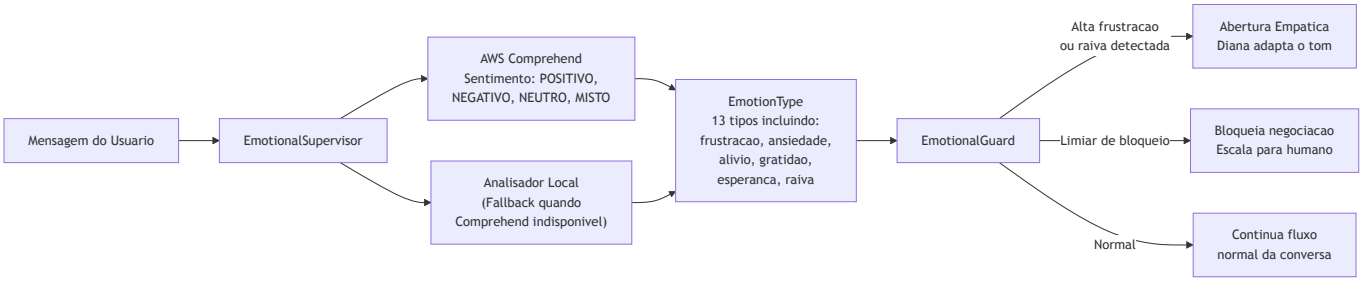
12.2 Resiliencia

Padrao	Implementacao	Servico
Retry com backoff	Polly `WaitAndRetryAsync`	Mesh (.NET)
Circuit breaker	Polly `CircuitBreakerAsync`	Mesh (.NET)
RetryPolicy	`RetryPolicy` nativo do LangGraph nos nodes	SmartCollect (Python)
Dead Letter Queue	Topico `smartcollect.dlq` com `DLQReprocessor`	Ambos
Producer idempotente	`KAFKA_ENABLE_IDEMPOTENCE=true`	SmartCollect
Deduplicacao de consumer	Rastreamento via `correlation_id + event_id`	Ambos
Cache de token	`RaaSTokenService` cacheia tokens OAuth2 ate expiracao	Mesh
LLM via Bedrock	Claude (Anthropic) via AWS Bedrock com retry nativo do SDK	SmartCollect
Checkpointer	Redis/MemorySaver para recuperacao de estado da conversa	SmartCollect

12.3 Seguranca e Conformidade

Preocupacao	Implementacao
Conformidade LGPD	Agente de Compliance valida tom e tratamento de dados em cada resposta
Mascaramento de PII	Mascaramento de CPF em logs (3 primeiros digitos visiveis, restante mascarado)
Row Level Security	RLS do PostgreSQL habilitado em todas as tabelas principais
Trilha de Auditoria	Tabela `audit_logs` com triggers automaticos em INSERT/UPDATE/DELETE
Whitelist de Numeros	`NumberGatewayService` filtra numeros WhatsApp permitidos
Autenticacao de API	OAuth2 para RaaS, chaves de API para Infobip e Telegram
Criptografia de Dados	TLS em transito, criptografia RDS em repouso
Aceite de Termos	PDF de termos servido via `/api/terms/use/` , rastreado no fluxo de acordo

12.4 Inteligencia Emocional



A analise emocional roda em cada mensagem do usuario e influencia:

- O tom e nivel de empatia da Diana
- Se deve prosseguir com a negociacao ou pausar
- Decisoes de escalacao para operadores humanos

12.5 RAG (Retrieval Augmented Generation)

Componente	Tecnologia	Finalidade
Repositorio de Documentos	`PostgresDocumentStore` + PGVector	Armazena documentos de FAQ, politicas e conhecimento com embeddings vetoriais
Embeddings	AWS Bedrock (1536 dimensoes)	Gera embeddings vetoriais para documentos e consultas
Busca por Similaridade	Indice IVFFlat (cosseno)	Encontra documentos relevantes para enriquecimento de contexto
Base de Conhecimento	AWS Bedrock Knowledge Base	Fonte RAG alternativa para colecoes maiores de documentos

O pipeline RAG e utilizado principalmente pelo Agente de FAQ, que enriquece os prompts LLM com documentos de politica relevantes, respostas de FAQ e diretrizes de negociacao antes de gerar respostas. O Agente de Negociacao tambem pode consultar a base de conhecimento para contextualizar ofertas.

Apendice A: Glossario

Termos de Dominio

Termo	Definicao
Acordo (Agreement)	Compromisso formalizado entre o cliente e a empresa para quitacao de uma divida
Arrangement	Materializacao tecnica de um Acordo no sistema RaaS
Cliente (Client)	Pessoa fisica ou juridica com divida ativa, identificada por CPF/CNPJ
Conversa (Conversation)	Sessao de interacao atraves de um canal de comunicacao
Divida (Debt)	Obrigacao financeira pendente com valor, juros e data de vencimento
Desconto (Discount)	Reducao no valor total da divida como incentivo ao pagamento
Parcela (Installment)	Fracao do valor total do acordo paga periodicamente
Intencao (Intent)	Proposito detectado na mensagem do usuario (saudacao, negociacao, pagamento, etc.)
Oferta (Offer)	Proposta de negociacao com opcoes de pagamento e descontos
Boleto	Guia de pagamento brasileiro gerado apos a criacao do acordo

Termos Tecnicos

Termo	Definicao
Agente Especializado	Entidade autonoma de IA com raciocinio proprio via LLM (Claude/Bedrock), system prompt exclusivo, tools dedicados e dominio especifico. Herda de `BaseAgent`
Agente Supervisor	Agente orquestrador que detecta intencao via LLM e delega para agentes especializados. Implementa o padrao Supervisor Multi-Agent
Arquitetura Multi-Agent	Padrao arquitetural onde multiplos agentes especializados de IA cooperam via estado compartilhado, orquestrados por um supervisor central
BaseAgent	Classe base abstrata que fornece capacidades de geracao LLM, composicao de prompts e deteccao de contexto a todos os agentes especializados
Bounded Context	Fronteira explicita para um modelo de dominio: Negociacao, Conversa, Cobranca
Checkpointer	Mecanismo do LangGraph para persistir estado compartilhado entre turnos de conversa
Correlation ID	UUID que rastreia uma mensagem ponta-a-ponta em todos os servicos
DLQ	Dead Letter Queue para mensagens que falharam no processamento
Gateway	Adaptador que conecta o sistema a servicos externos
MCP	Model Context Protocol: interface padrao para acesso a tools por agentes de IA
RAG	Retrieval Augmented Generation: combinacao de busca de documentos com geracao por LLM, utilizado pelo Agente de FAQ
RaaS	Recovery as a Service: backend financeiro da emDia para gestao de dividas
StateGraph	Infraestrutura de orquestracao do LangGraph que coordena a execucao dos agentes especializados

Metricas de Negocio

Metrica	Definicao
Taxa de Conversao	Percentual de conversas que resultam em acordo assinado
Tempo Medio de Negociacao	Tempo desde o inicio da conversa ate o fechamento do acordo
Percentual de Desconto	Desconto medio concedido nos acordos
Taxa de Recuperacao	Percentual de acordos efetivamente pagos

Estados do Sistema

Conversa: `active -> completed | abandoned | escalated`

Acordo: `pending -> accepted -> completed | rejected | expired | cancelled`

Pagamento: `pending -> processing -> completed | failed | cancelled`

Apendice B: Evolucao Phoenix

SmartCollect Phoenix e uma reescrita completa planejada do motor Multi-Agent de IA, mantendo 100% de compatibilidade com os contratos externos (topicos Kafka, tools MCP, schema PostgreSQL, integracao Mesh). A evolucao transforma agentes baseados em classes (`BaseAgent`) para agentes baseados em funcoes puras com middleware stack.

Comparacao V1 vs Phoenix

Aspecto	V1 (Atual)	Phoenix (Planejado)
Estado	`conversation_history` + `messages` (dual)	Apenas `messages` com reducer `add_messages`
Roteamento	`add_conditional_edges` + FSM	`Command(update={...}, goto=target)` atomico
Agentes	Classes especializadas herdando de `BaseAgent`	Funcoes puras `async def agent(state, config)` com middleware
Agente de Negociacao	14+ classes handler + `NegotiationStateMachine`	Sub-grafo com 9 agentes, `interrupt()` , rollback via `Command(goto=...)`
API de Invoke	`dict` sem tipagem	`version="v2"` com `GraphOutput.value` + `.interrupts`
LLM	`LLMFactory` com AWS Bedrock	`ModelFallbackMiddleware` (built-in) via Bedrock
Sumarizacao	`ContextTrimmer` manual por agente	`SummarizationMiddleware` (automatico)
Injecao de Prompt	`DynamicPromptBuilder.build_prompt()` por agente	Middleware `@dynamic_prompt` (automatico)
Guardrails	Classe custom invocada por agente	`SmartCollectGuardrailMiddleware` (hook `after_model`)
PII	Nao implementado	`PIIMiddleware` para mascaramento de CPF
Resiliencia	Try/except manual	`RetryPolicy` nativo em nodes com API
Telemetria	`TelemetryPublisher` injetado	`get_stream_writer()` nativo
Python	3.9 compat (`Optional[X]`)	3.12 nativo (`X

Documentos de Especificacao Phoenix

Documento	Conteudo
`spec/00-OVERVIEW.md`	Principios, stack, diagrama de alto nivel
`spec/01-ARCHITECTURE.md`	Estado, contexto, grafo, middleware stack
`spec/02-NODES.md`	Especificacoes dos agentes (entrada/saida/logica)
`spec/03-CONTRACTS.md`	Contratos Kafka, MCP, PostgreSQL, API
`spec/04-DOMAIN.md`	Regras de negocio, CPF, intencao, ofertas
`spec/05-PROMPTS.md`	Templates de prompt
`spec/06-EXECUTION-PLAN.md`	Plano de execucao em 6 fases
`spec/07-SCALABILITY.md`	Escalabilidade horizontal, particionamento
`spec/08-CONTEXT-ENGINEERING.md`	Sumarizacao, Store API, recuperacao JIT
`spec/09-LANGGRAPH-PATTERNS.md`	Command, interrupt, v2, middleware
`spec/10-NEGOTIATION-SUBGRAPH.md`	Sub-grafo de 9 agentes de negociacao com rollback
`spec/11-EMOTIONAL-INTELLIGENCE.md`	Pipeline Comprehend + analisador local
`spec/12-GUARDRAILS.md`	Middleware de compliance, PII
`spec/13-OBSERVABILITY.md`	Telemetria, metricas, limpeza, DLQ

Apendice C: Radar Tecnologico

Tecnologia	Versao	Categoria	Status	Finalidade
Python	3.12	Linguagem	Adotar	Runtime do motor de IA
.NET	10.0	Framework	Adotar	Gateway de integracao
LangGraph	>=1.1.2	Framework IA	Adotar	Orquestracao Multi-Agent
LangChain	>=1.1.0	Framework IA	Adotar	Middleware, abstracao de LLM
FastAPI	>=0.104.0	Framework Web	Adotar	API REST Python
ASP.NET Core	10.0	Framework Web	Adotar	API REST .NET
Apache Kafka	7.5.0 (Confluent)	Mensageria	Adotar	Backbone event-driven
PostgreSQL	15	Banco de Dados	Adotar	Armazenamento principal
PGVector	(extensao pg15)	BD Vetorial	Adotar	Embeddings RAG
Redis	7	Cache/Estado	Adotar	Checkpointer, cache
AWS Bedrock	-	Servico IA	Adotar	LLM unico (Claude Anthropic via Bedrock)
AWS Comprehend	-	Servico IA	Adotar	Analise de sentimento e emocoes
Infobip	-	Comunicacao	Adotar	Envio WhatsApp/SMS/Email
Telegram Bot API	-	Comunicacao	Adotar	Canal Telegram
Docker	-	Container	Adotar	Desenvolvimento local
Kubernetes (EKS)	-	Orquestracao	Adotar	Deploy em producao
Amazon MSK	-	Kafka Gerenciado	Adotar	Kafka em producao
Amazon RDS	-	BD Gerenciado	Adotar	PostgreSQL em producao
Amazon ElastiCache	-	Cache Gerenciado	Adotar	Redis em producao
Polly	-	Resiliencia	Adotar	Retry/circuit breaker .NET
Serilog	10.0.0	Logging	Adotar	Logging estruturado .NET
LangSmith	-	Observabilidade	Experimentar	Rastreamento e depuracao de LLM
Pydantic	>=2.0.0	Validacao	Adotar	Validacao de schemas
httpx	>=0.25.0	Cliente HTTP	Adotar	Chamadas MCP async
aiokafka	>=0.10.0	Cliente Kafka	Adotar	Consumer/producer Kafka async
boto3	>=1.28.0	AWS SDK	Adotar	Bedrock, Comprehend